

A More examples

// Example 3: Matrix operations with checked dimensions

```
case class Matrix(width: Pos, height: Pos):  
  private val size = width * height  
  private val data = CheckedArray(size.runtimeChecked)  
  def index(i: Pos with i < height,  
            j: Pos with j < width): {res: Pos with res < data.size} =  
    (i * width + j).runtimeChecked  
  def apply(i: Pos with i < height, j: Pos with j < width): Double =  
    data(index(i, j))  
  def update(i: Pos with i < height, j: Pos with j < width, x: Double): Unit =  
    data(index(i, j)) = x  
  def transpose(): {v: Matrix with v.width == height && v.height == width} = ???  
  def mul(that: Matrix with that.height == width):  
    {v: Matrix with v.width == that.width && v.height == height}  
  =  
    val res = Matrix(that.width, height)  
    forRange(0, height): i =>  
      forRange(0, that.width): j =>  
        var sum = 0.0  
        forRange(0, width): k =>  
          sum = sum + this(i, k) * that(k, j)  
        res(i, j) = sum  
    res  
def matricesExample =  
  val m1 = Matrix(2, 3)  
  val m2 = Matrix(3, 2)  
  m1.mul(m2)  
  val m1T = m1.transpose()  
  m1T.mul(m1)
```

// Example 4: Memoized Fibonacci with a dependent map

```
case class DepMap[K, V](p: (K, V) => Boolean):  
  def put(n: K, v: V with p(n, v)): Unit = ???  
  def get(n: K): Option[{res: V with p(n, res)}] = ???  
def fib(n: Int): {r: Int with r == (if n <= 1 then 1 else fib(n - 1) + fib(n - 2))} =  
  if n <= 1 then 1 else fib(n - 1) + fib(n - 2)  
val cache: (DepMap[Int, Int] with (cache == DepMap[Int, Int]((k, v) => v == fib(k)))) =  
  DepMap[Int, Int]((k, v) => v == fib(k))  
def fibMemo(n: Int): {res: Int with res == fib(n)} =  
  cache.get(n) match  
    case Some(res) => res  
    case None =>  
      val res: (Int with (res == fib(n))) =  
        if n <= 1 then 1 else fibMemo(n - 1) + fibMemo(n - 2)  
      cache.put(n, res)  
      res
```